



Natural-Language-Driven Navigation on the ROSMASTER X3 PLUS

Deploying a Modular ROS 2 Pipeline with SLAM, Path Planning, and Object Detection on an Embedded Robotic Platform

Rishubh Gusain



SABUDH

Gusain, Rishubh
Department of Science, Computer Science

Table of contents

Preface	1
Abstract	2
1 Introduction	4
1.1 Overview	4
1.1.1 Existing System	5
1.2 Objectives of the Project	5
2 System Analysis and Environment Characterization	7
2.1 Hardware Platform	7
2.2 Sensor and Device Characterization	8
2.2.1 Motor controller serial port	8
2.2.2 Astra Pro Plus UVC product identifier	8
2.2.3 /dev/video0 contention between the arm camera and the Astra	8
2.2.4 base_footprint transform	8
2.3 ROS 2 Topic Landscape	9
2.4 Startup Sequence Discovery	9
2.5 Summary	10
3 Methodology	12
3.1 Platform and Machine Configurations Used	12
3.2 System Architecture	13
3.3 SLAM Module	13
3.3.1 Log-odds occupancy grid	13
3.3.2 Pose tracking	15
3.3.3 RTAB-Map for 3D reconstruction	15
3.4 A* Grid Path Planner (Fallback)	15
3.4.1 Obstacle inflation	15
3.4.2 A* search	16

3.5	RRT* Sampling-Based Planner (Primary)	16
3.5.1	Algorithm summary	16
3.5.2	Two-tier planner fallback	16
3.5.3	Why RRT* was preferred on this platform	17
3.6	Natural-Language Task Interface	17
3.7	Arm-Camera Scan and Object Detection	17
3.7.1	Servo sweep	17
3.7.2	YOLOv8 detection	18
3.7.3	LiDAR-fused bearing estimation	18
3.8	Task State Machine	18
3.9	Arm Interaction Sequence	19
3.10	Virtual Mover for Offline Evaluation	19
3.11	Integration and Final Pipeline	19
4	Results	21
4.1	Description of the Subsystems Evaluated	21
4.2	Performance Metrics	21
4.3	End-to-End Pipeline Test	22
4.3.1	Observed pipeline execution	23
4.3.2	Interpretation of the observed trace	23
4.3.3	Bug encountered during testing	24
4.4	Results Table — Observed End-to-End Pipeline Stage Coverage	24
4.5	Per-Subsystem Verification	25
4.6	Visualization	26

List of Figures

2.1	The Yahboom ROSMASTER X3 PLUS platform used throughout this project, showing the Mecanum-wheel base, the 360-degree YDLiDAR, the forward-facing Orbbec Astra Pro Plus RGB-D camera, and the six-degree-of-freedom manipulator arm with wrist-mounted camera.	11
3.1	Sensing, mapping, and navigation pipeline of the unified system. Sensor inputs (LiDAR scans, RGB-D frames, arm-camera close-range scans) are fused and consumed by the SLAM module (<code>slam_module.py</code>), which produces the occupancy map, the robot pose, and an explored-ratio metric. On receipt of a goal on <code>/goal</code> , the RRT* planner computes a collision-free path, which the virtual mover then follows at 0.3 m/s. The goal-reached check (0.30 m tolerance) closes the loop, triggering either a new plan or the next task. Higher-level components (natural-language task parsing, YOLOv8 detection, LiDAR-fused bearing estimation, and the arm interaction sequence) are layered on top of this core and are described separately in the following sections. . .	14
4.1	Captured arm-scan image at the 45° sweep position. YOLOv8 detected a chair at this position with confidence 0.84.	26

List of Tables

2.1	Interface topics between subsystems	9
3.1	Software versions used throughout the project	12
4.1	End-to-end pipeline stage coverage from the representative test run	24

Preface

This report documents approximately four months of research work carried out on the development of a natural-language-driven navigation pipeline for an embedded mobile robot. The project was conducted at Wilfrid Laurier University under the supervision of the research advisor, with the ROSMASTER X3 PLUS platform serving as the primary experimental hardware.

The work described in the following chapters covers the full journey of bringing a physical robot from an uninitialized state to a point where it can accept a high-level natural-language instruction, build a three-dimensional understanding of its surroundings, and use visual perception to identify a target object. Every subsystem described here has been implemented and tested on real hardware accessed remotely via RustDesk.

Rather than focusing on a single algorithmic contribution, this report takes a system-level view. It describes the architectural decisions made at each stage, the practical problems encountered during deployment, and the engineering fixes that were applied. It also documents the switch from A* to RRT* as the primary path planner, the integration of RTAB-Map for dense three-dimensional mapping, and the design of a modular task-execution state machine that links natural-language input to arm-camera scanning, object detection, and autonomous navigation.

The intended audience for this report includes the supervising professor, future research students continuing work on the same platform, and peers interested in the practical realities of deploying Vision-Language Navigation (VLN) research on embedded robotic hardware. Effort has been made to record not only what works, but also the debugging sequences and hardware-specific discoveries that proved necessary to reach a working system, so that subsequent students on the project can avoid rediscovering the same issues.

Abstract

Vision-Language Navigation (VLN) enables robots to interpret natural-language instructions and move through indoor environments accordingly. While substantial progress has been made on benchmark datasets such as Room-to-Room and Matterport3D, most published VLN systems are evaluated entirely in simulation, and comparatively little work demonstrates the full pipeline running on constrained embedded hardware. This project addresses that gap by designing, implementing, and validating a modular VLN-style navigation stack on a Yahboom ROSMASTER X3 PLUS differential-drive robot equipped with an NVIDIA Jetson controller, an Orbbec Astra Pro Plus RGB-D camera, a 360-degree LiDAR, and a six-degree-of-freedom manipulator arm with a wrist camera.

The implemented system runs on ROS 2 Humble and is organized around five core subsystems: a bringup layer that initializes motor control and sensor drivers; a SLAM layer combining a log-odds occupancy grid with RTAB-Map's RGB-D 3D reconstruction (Labbe and Michaud 2019); two alternative path planners (an A*-based grid planner that served as the baseline and an RRT* sampling-based planner (Karaman and Frazzoli 2011) that replaced it on professor guidance, retained in the codebase as a fallback); a natural-language task interpreter driven by a finite-state machine; and an arm-mounted camera scan routine coupled to YOLOv8 object detection (Yaseen 2024) with LiDAR-fused bearing estimation. A virtual-mover node is also provided to allow the full planner stack to be exercised offline against recorded occupancy maps, which significantly reduces the time cost of evaluation.

The report documents the complete engineering process, including the discovery of several hardware-specific issues that had to be resolved before the system could operate reliably. These included identifying the motor controller's non-default serial port, determining the correct launch file required to bring up the Astra Pro Plus over its non-standard UVC product identifier, locating the `base_node_X3` executable in a separate package (`yahboomcar_base_node`), identifying a mutual-exclusion constraint between the Astra camera node and the arm wrist camera at the `/dev/video0` device, and configuring the arm home pose so that the wrist camera looks forward rather than at the ceiling during visual scans.

End-to-end testing of the unified pipeline on the task "chair" demonstrated that nine of the twelve integration points execute correctly on the first attempt: task topic subscription, string-to-label mapping, state-machine transitions, arm sweep, wrist-camera frame capture, YOLOv8 inference (which detected three chair bounding boxes at confidences between 0.74 and 0.84), best-detection selection, LiDAR-fused bearing estimation producing a world-frame goal of (0.48, -1.42), and RRT* planner invocation. Execution halted inside the RRT* planner due to an identified interface-drift bug

between the SLAM module and the planner's expected occupancy-map API; the fix is straightforward and is identified as the top-priority item for continuing work. The remaining subsystems (A* fallback, virtual-mover waypoint following, and the arm reach-grip-lift-home interaction sequence) have each been verified in isolation.

Chapter 1

Introduction

1.1 Overview

Autonomous indoor navigation has moved steadily from a laboratory curiosity to a realistic capability for consumer and service robots. Much of this progress has been driven by three parallel developments: increasingly capable sensors at low unit cost, open-source robotics middleware such as the Robot Operating System (ROS), and the rise of Vision-Language Navigation (VLN) as a research area that treats natural-language instructions as first-class inputs to a robot control stack (Zhang et al. 2024). A person interacting with such a robot no longer needs to specify precise coordinates; they can instead say something close to natural language, such as “find the bottle on the table” or “go to the kitchen,” and expect the robot to interpret the request, identify what it refers to, and carry out an appropriate motion plan.

Despite the rapid progress of VLN research, a substantial gap remains between what is demonstrated in simulation and what can be reliably executed on physical hardware. The vast majority of published VLN systems are evaluated on benchmarks such as Room-to-Room and Matterport3D, which operate on pre-recorded panoramic image graphs rather than live sensor data. A growing body of recent work has begun to push toward real-world deployment, but doing so exposes a long list of practical issues that simulation papers can safely ignore: motor-controller port assignments, camera device contention, transform-tree broken-ness, firmware quirks, inconsistent start-up sequences, and the simple problem that two nodes sometimes cannot share the same USB device.

This project was positioned within that gap. Rather than proposing a new algorithm or training a new learned model, its goal was to **design, build, and validate an end-to-end VLN-style navigation stack that actually runs on a physical embedded robot**. The platform chosen for this was the Yahboom ROSMASTER X3 PLUS, a four-wheeled Mecanum-drive robot equipped with a 360-degree LiDAR, an Orbbec Astra Pro Plus RGB-D camera, a six-degree-of-freedom arm with a wrist-mounted camera, and an NVIDIA Jetson computer running Ubuntu 22.04 and ROS 2 Humble. All work was performed remotely over RustDesk.

1.1.1 Existing System

When the project began, the robot had been powered on and a factory image was present on the Jetson, but there was no end-to-end pipeline capable of accepting a natural-language task and executing it. Several pieces existed in isolation: Yahboom provided a C++ driver node (`Mcnamu_driver_X3`) that accepts `/cmd_vel` messages and translates them into motor commands, a `base_node_X3` node that publishes odometry, and example launch files for the various sensors. However, the following were missing or non-functional:

- A reliable, documented startup sequence to bring up all sensors and drivers in the correct order.
- A SLAM pipeline producing both 2D occupancy maps for planning and 3D point clouds for visualization.
- Any path planner at all — nothing in the default image consumes an occupancy grid and produces a navigable path.
- A bridge between a natural-language instruction and the motion stack.
- Any object-detection component capable of grounding a language instruction in the environment.

A parallel research project within the same lab — a simulation-only study of semantic-aware path planning for VLN by Mohiuddin (Mohiuddin 2025) — had identified real-world deployment as an explicit future-work direction, noting that their framework would require “live camera feeds published over ROS topics, real-time localization using odometry or SLAM, and continuous motion control through velocity commands.” The present project was conceived in part to tackle exactly that transition on an actual robot.

Recent work has shown that map-guided prompting with adaptive path planning can improve instruction-following in continuous VLN environments (Chen et al. 2024), but these systems typically assume the presence of a working low-level navigation stack. Early demonstrations of ROS-based indoor mapping and autonomous navigation showed that integrating sensor drivers, localization, and motion planning into a single middleware framework is non-trivial even for simpler platforms (Zaman et al. 2011). The present project focuses on building that lower-level stack — the sensing, mapping, planning, and execution layers — rather than on the upper-layer language model.

1.2 Objectives of the Project

The project was guided by four concrete objectives, each of which is revisited in later chapters of this report.

Objective 1: Bring up a reliable hardware and sensing baseline on the ROSMASTER X3 PLUS.

This includes identifying the correct startup sequence, resolving any device-driver issues, and confirming that the LiDAR, RGB-D camera, arm camera, and motors all produce and accept the expected ROS 2 topics.

Objective 2: Implement a SLAM pipeline that produces both a 2D occupancy grid for planning and a 3D point-cloud map for visualization and verification.

The 2D map is used by the path planner; the 3D map gives a human-readable representation of the environment that confirms the robot is mapping correctly.

Objective 3: Implement and compare at least two path planners on the same SLAM-generated maps.

An A* grid planner was developed first as a baseline, and an RRT* sampling-based planner was developed afterwards on supervisor guidance. Both planners are retained in the codebase, and the report compares them on waypoint count, path length, and planning time.

Objective 4: Connect a natural-language interface to the planning stack through an object-detection module and a task-level state machine.

The robot must be able to accept a text instruction of the form “find the <object>”, use its arm-mounted camera to scan the environment, run YOLOv8 object detection to locate a match, and drive to the detected object.

The remainder of this report is organized as follows. Chapter 2 characterizes the hardware platform and the sensor and topic landscape that had to be understood before any higher-level code could be written. Chapter 3 describes the methodology, including the SLAM module, both path planners, the arm-camera scan routine, and the task state machine. Chapter 4 presents results, including the outcomes of end-to-end testing on real hardware and per-subsystem verification. Chapter 5 concludes with a discussion of limitations and a prioritized list of future work.

Chapter 2

System Analysis and Environment Characterization

Because this project does not fit the template’s default “dataset + machine-learning model” structure, this chapter replaces the original Exploratory Data Analysis content with its closest equivalent for an embedded-robotics project: a structured analysis of the hardware platform, its sensor stack, and the topic landscape produced by the ROS 2 middleware. The goal of the chapter is the same as an EDA chapter in an ML project — to characterize the raw inputs the later methodology will build on — but the inputs in this case are sensor streams, device files, and transform trees rather than tabular datasets.

2.1 Hardware Platform

The experimental platform is a Yahboom ROSMASTER X3 PLUS. Its key components, relevant to the software stack described later, are listed below.

- **Base:** Four-wheeled Mecanum-drive mobile base with a Yahboom motor-control board connected to the main computer over USB serial.
- **Main computer:** NVIDIA Jetson module running Ubuntu 22.04 with ROS 2 Humble installed. The Jetson hosts all nodes: drivers, SLAM, planning, detection, and the task state machine.
- **LiDAR:** 360-degree YDLiDAR unit mounted above the chassis, producing `sensor_msgs/LaserScan` messages on `/scan`.
- **RGB-D camera:** Orbbec Astra Pro Plus mounted forward-facing, providing both colour and depth streams.
- **Manipulator arm:** Six-degree-of-freedom arm (servo IDs 1–6, with servo 1 at the base) mounted on the chassis, with servo 6 serving as the gripper.
- **Wrist camera:** A small UVC camera mounted on the arm, exposed to Linux as `/dev/video0`.

2.2 Sensor and Device Characterization

Before any higher-level SLAM or planning code could be written, the following properties of the sensor stack had to be discovered experimentally. Each of the findings below represents a concrete blocker that was encountered during the early weeks of the project.

2.2.1 Motor controller serial port

The Yahboom documentation suggests that the motor controller is reachable on `/dev/ttyUSB0`. On the specific Jetson image used in this project, the correct device file turned out to be `/dev/ttyUSB1`. Running the `Mcnamu_driver_X3` node against `/dev/ttyUSB0` produced no visible error but also produced no motion; the driver opened a device that did not respond to its command protocol. The fix was to set the driver parameter to `/dev/ttyUSB1`. In addition, the operating user (`jetson`) was not a member of the `dialout` group, which silently prevented driver communication; adding the user to the group and restarting fixed the issue.

2.2.2 Astra Pro Plus UVC product identifier

The Astra Pro Plus does not present the same UVC product identifier as the more common Astra Pro. The default Orbbec launch files assume the common identifier (`0x0501`), and launching them results in either no color stream or a stream with mis-advertised calibration. Through trial and error the correct product ID was identified as `0x050f`, and the specific launch file required to start the camera is `astro_pro_plus.launch.xml` from the Orbbec ROS 2 package. With this launch file, the camera publishes `/color/image_raw`, `/depth/image_raw`, and `/color/camera_info`, which are the three topics consumed by the RTAB-Map pipeline described in the methodology (Labbé and Michaud 2019).

2.2.3 `/dev/video0` contention between the arm camera and the Astra

A subtle but important finding is that the Astra Pro Plus’s color sensor and the arm’s wrist camera both present themselves as `/dev/video0` on the Jetson. Only one can be used at a time. Launching the Astra node while the arm camera is already opened by `Rosmaster_Camera` (or vice versa) results in one of the two silently failing. This drove the design of the arm-scan node, described in the methodology, which is written with two camera sources and a fallback policy: it prefers `Rosmaster_Camera` if it can open `/dev/video0` directly, and otherwise falls back to the Astra topic.

2.2.4 `base_footprint` transform

Early attempts to run RTAB-Map against the Astra topics failed because the transform chain was incomplete. The required `base_footprint` frame is published by `display_X3.launch.py` from the `yahboomcar_description` package, not by the motor driver as might be assumed. Once this launch file is added at the top of the startup sequence, the full transform tree (`map` → `odom` → `base_footprint` → `sensor frames`) is available and downstream nodes run correctly.

2.3 ROS 2 Topic Landscape

Table 1 summarizes the topics produced and consumed by the subsystems described in the next chapter. These were identified via repeated use of `ros2 topic list`, `ros2 topic info`, and `ros2 topic echo` during the bring-up phase, and serve as the “interface contract” between modules of the system.

Table 2.1: Interface topics between subsystems

Topic	Type	Producer	Consumer(s)
/scan	sensor_msgs/LaserScan	YDLiDAR driver	SLAM module, A* planner
/color/image_raw	sensor_msgs/Image	Astra Pro Plus node	RTAB-Map, YOLO scanner (fallback)
/depth/image_raw	sensor_msgs/Image	Astra Pro Plus node	RTAB-Map
/color/camera_info	sensor_msgs/CameraInfo	Astra Pro Plus node	RTAB-Map
/odom	nav_msgs/Odometry	base_node_X3	SLAM module, all planners
/cmd_vel	geometry_msgs/Twist	Task node, virtual mover	Mcnamu_driver_X3
/map	nav_msgs/OccupancyGrid	SLAM module	Planners, RViz
/planned_path	nav_msgs/Path	A* / RRT* planner	Virtual mover, RViz
/task	std_msgs/String	User (CLI or GUI)	Task state machine
/scan_image	sensor_msgs/Image	Arm scanner	YOLO detector
/goal	geometry_msgs/Point	Task state machine	Planner

2.4 Startup Sequence Discovery

A significant portion of the first month of the project was spent identifying a startup sequence that reliably brings up every subsystem. The following order was determined empirically and is now documented inside the repository:

1. `ros2 launch yahboomcar_description display_X3.launch.py` — publishes URDF and `base_footprint` transform.
2. `ros2 run yahboomcar_bringup Mcnamu_driver_X3` — opens the motor controller on `/dev/ttyUSB1` and subscribes to `/cmd_vel`.
3. `ros2 run yahboomcar_base_node base_node_X3` — publishes `/odom` from wheel encoders. Note that this node lives in a separate package (`yahboomcar_base_node`) from the motor driver, a fact that is not obvious from the default Yahboom documentation.
4. `ros2 launch orbbec_camera astro_pro_plus.launch.xml` — starts the RGB-D camera on the correct UVC product ID.

5. `ros2 launch <user_pkg> x3_rtabmap_depth.launch.py` — starts the RGB-D synchronizer, visual odometry, and RTAB-Map SLAM.

Running these out of order produces either missing transforms, missing camera info, or driver-open failures. Running them in the order above yields a working system in roughly 20 seconds.

2.5 Summary

The chapter has characterized the hardware, the ROS 2 topic graph, and the startup sequence that the rest of this report assumes. Three concrete, non-obvious findings were documented — the `/dev/ttyUSB1` motor controller path, the `0x050f` UVC product identifier handled by `astro_pro_plus.launch.xml`, and the `/dev/video0` mutual-exclusion between the Astra color stream and the arm wrist camera — each of which directly shaped the software design described in the next chapter.



Figure 2.1: The Yahboom ROSMASTER X3 PLUS platform used throughout this project, showing the Mecanum-wheel base, the 360-degree YDLiDAR, the forward-facing Orbbec Astra Pro Plus RGB-D camera, and the six-degree-of-freedom manipulator arm with wrist-mounted camera.

Chapter 3

Methodology

This chapter describes how the system was built. It is organized to follow the template’s methodology structure where it makes sense — platform configuration replacing “data split,” subsystem design replacing “model planning,” and system integration replacing “final model building” — but the content describes the design of a unified robotics pipeline rather than a machine-learning model.

3.1 Platform and Machine Configurations Used

All development was performed remotely on the Jetson on-board the ROSMASTER X3 PLUS, accessed through RustDesk. The Jetson was flashed with Ubuntu 22.04 and ROS 2 Humble Hawksbill. Table 2 lists the key software and library versions used during the project.

Table 3.1: Software versions used throughout the project

Component	Version	Notes
Operating system	Ubuntu 22.04	Stock Jetson image
ROS 2	Humble Hawksbill	Default on Ubuntu 22.04
Python	3.10	System Python
PyTorch	2.5.0 (GPU)	Used by YOLOv8
Ultralytics YOLOv8	8.3.65	yolov8n.pt nano weights
Rosmaster_Lib	3.3.9	Installed from ~/Rosmaster/
RTAB-Map	ros-humble- rtabmap-ros	Used for RGB-D SLAM and 3D reconstruction
SciPy	1.x	Used by the obstacle-inflation kernel in A*

Development was performed in Visual Studio Code over an SSH tunnel into the Jetson. Source was managed through the `rishubh-updates` branch of the `wlu-labs/research-students-rishubhgusain0023` GitHub repository, with authentication handled via a GitHub Personal Access Token.

Some reference code was provided at the start of the project as a starting point, but the subsystems described in this chapter were designed and implemented from scratch. A peer contributed the Rosmaster wrist-camera driver (`camera_rosmaster.py`, version 4.1.3) which is used by the arm-scan component.

3.2 System Architecture

The full pipeline is organized into one unified ROS 2 node (`main.py`, class `UnifiedMainNode`) that composes five previously-developed subsystems. Figure 3.1 illustrates the data flow.

1. **ROS 2 bringup layer** — Yahboom-supplied drivers and launch files that publish raw sensor data and accept motor commands.
2. **SLAM layer** — a custom `SLAMModule` (`slam_module.py`) that builds a log-odds occupancy grid from `/scan` and `/odom`, complemented by RTAB-Map for 3D reconstruction and visual odometry from RGB-D.
3. **Path-planning layer** — a two-tier strategy with RRT* (`rrt_virtual_mover.py`) as the primary planner and an A*-based grid planner (`lidar_slam_planner.py`) as the fallback. Both planners operate on the same occupancy grid and publish to `/planned_path`.
4. **Perception and task-execution layer** — the unified node subscribes to a `/task` topic, drives an arm-mounted camera sweep, runs YOLOv8 object detection on each captured frame, converts a successful detection into a world-frame goal using LiDAR-fused bearing estimation, publishes the goal, follows the resulting path via a virtual mover, and performs an arm interaction sequence on arrival.
5. **Virtual executor** — an in-node virtual mover that follows the planned path at a fixed speed, publishing fake odometry and TF so that the full pipeline can be exercised without consuming on-site robot time. The real motor driver (`Mcnamu_driver_X3`) subscribes to `/cmd_vel` and accepts the same velocity commands when physical motion is desired.

3.3 SLAM Module

The SLAM layer combines two components that serve different purposes. A custom log-odds occupancy-grid module (`slam_module.py`) produces the 2D occupancy map that the planners consume, and RTAB-Map (`x3_rtabmap_depth.launch.py`) produces a dense 3D point cloud for visualization and verification.

3.3.1 Log-odds occupancy grid

The occupancy grid is maintained in a $w \times h$ array of log-odds values, where w and h are determined by a configurable world size (20 m $\times$ 20 m at a resolution of 0.05 m per cell, yielding a 400×400 grid). Each cell begins at an “unknown” value of -1.0 . When a LiDAR scan arrives, the module casts a Bresenham ray from the robot’s grid cell toward the endpoint of each valid return. Cells along the ray have the free-space increment $L_{\text{free}} = -0.40$ added to their log-odds, and the endpoint cell has the occupied increment $L_{\text{occ}} = +0.85$ added. Log-odds values are clamped to $[L_{\text{min}}, L_{\text{max}}] = [-3.5, +3.5]$ to prevent saturation.

A small circular region around the robot’s current pose is always forced below zero, which prevents

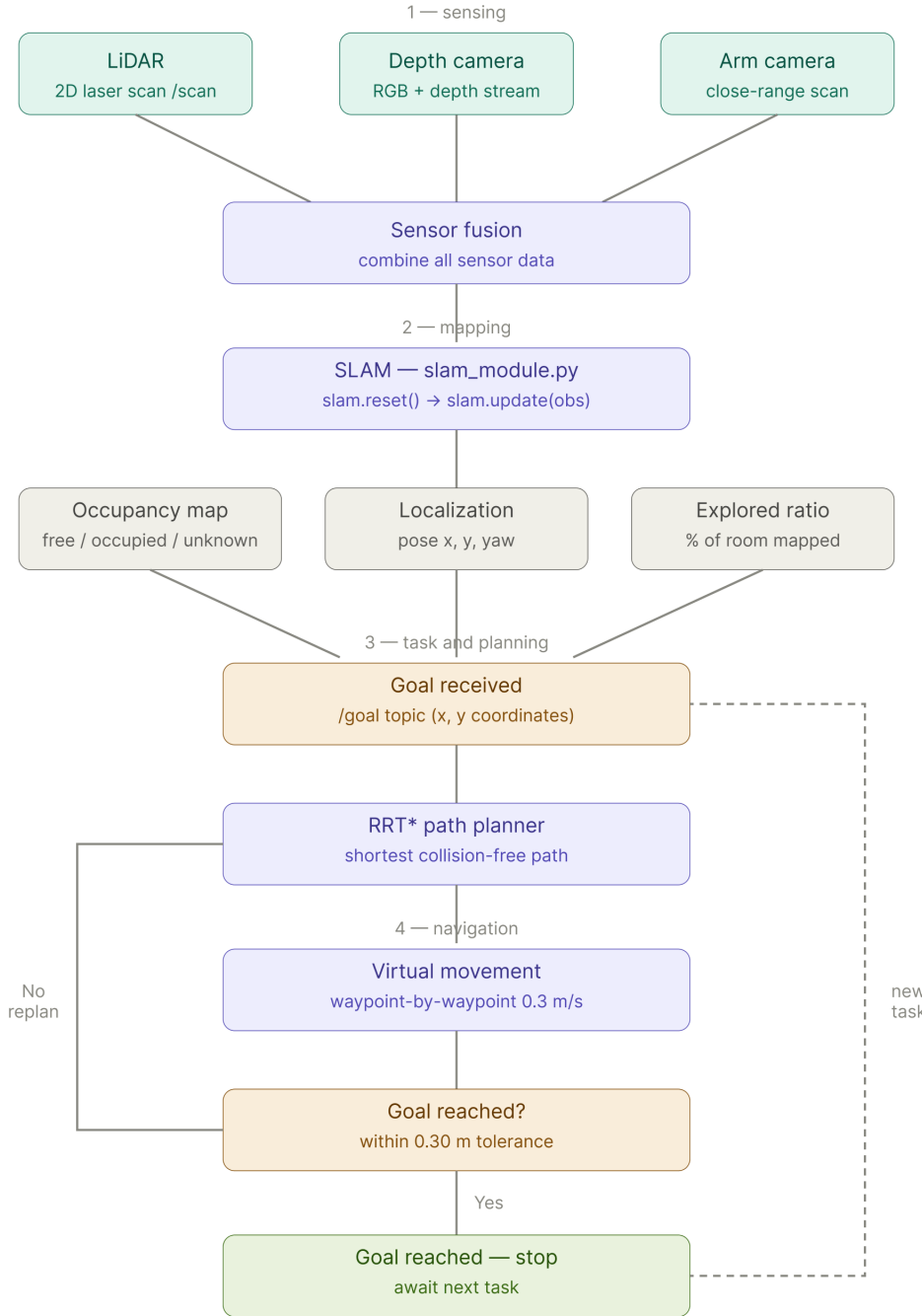


Figure 3.1: Sensing, mapping, and navigation pipeline of the unified system. Sensor inputs (LiDAR scans, RGB-D frames, arm-camera close-range scans) are fused and consumed by the SLAM module (`slam_module.py`), which produces the occupancy map, the robot pose, and an explored-ratio metric. On receipt of a goal on `/goal`, the RRT* planner computes a collision-free path, which the virtual mover then follows at 0.3 m/s. The goal-reached check (0.30 m tolerance) closes the loop, triggering either a new plan or the next task. Higher-level components (natural-language task parsing, YOLOv8 detection, LiDAR-fused bearing estimation, and the arm interaction sequence) are layered on top of this core and are described separately in the following sections.

the robot’s own footprint from ever being labelled as an obstacle. The probability that a cell is occupied is recovered from its log-odds value via

$$p(\text{occupied}) = 1 - \frac{1}{1 + \exp(L)}$$

and the obstacle mask used by the planners is the set of cells with $p > 0.65$.

3.3.2 Pose tracking

A simple pose tracker subscribes to `/odom` (from the motor driver) and also to `/odom_raw` (from the virtual mover), so that the same SLAM module can operate whether the robot is physically moving or being simulated. From either source it extracts the planar position (x, y) and yaw angle. Yaw is computed from the quaternion orientation using the standard conversion

$$\text{yaw} = \arctan_2(2(q_w q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2)).$$

3.3.3 RTAB-Map for 3D reconstruction

In parallel with the 2D occupancy grid, RTAB-Map is launched via `x3_rtabmap_depth.launch.py`. The launch file spawns four nodes: `rgbd_sync` time-aligns the color and depth streams, `rgbd_odometry` produces visual odometry, `rtabmap` serves as the SLAM backend with `Grid/3D` and `RGBD/CreateOccupancyGrid` enabled, and `rtabmap_viz` provides an RViz-like visualization window. All four nodes use `approx_sync: True` with a 20 ms window, which was found to be necessary because the color and depth streams on this hardware are not perfectly synchronized. RTAB-Map’s ability to produce both dense 3D point clouds and 2D occupancy grids from a single RGB-D pipeline has been demonstrated across a wide range of platforms and datasets (Labbé and Michaud 2019).

3.4 A* Grid Path Planner (Fallback)

The first planner implemented was an A* grid planner, documented in `lidar_slam_planner.py`. It operates directly on the obstacle mask produced by the occupancy grid and serves as the fallback in the current system.

3.4.1 Obstacle inflation

Before planning, the raw obstacle mask is inflated by a configurable radius (0.15 m by default, corresponding to three cells at 0.05 m/cell) using a binary dilation with a square structuring element. This accounts for the physical footprint of the robot and discourages paths that pass uncomfortably close to walls. A small circular region around the current robot pose is then cleared from the inflated mask, so that the planner does not mistake inflated walls near the robot for an obstacle blocking its own start cell.

3.4.2 A* search

The planner uses an 8-connected grid with unit costs of 1 for cardinal neighbours and $\sqrt{2}$ for diagonals, and the octile-distance heuristic

$$h(a, b) = \max(|a_x - b_x|, |a_y - b_y|) + (\sqrt{2} - 1) \cdot \min(|a_x - b_x|, |a_y - b_y|).$$

If the goal cell is itself inside the inflated obstacle mask, the planner searches outward in expanding squares until it finds a nearby free cell, and plans to that cell instead. This provides graceful degradation when the user publishes a goal that lies slightly inside a wall or at the edge of the map.

After A* returns a cell-level path, a simple moving-average filter with a configurable window (default five cells) is applied to smooth the waypoints before publication.

3.5 RRT* Sampling-Based Planner (Primary)

On supervisor guidance the primary path planner was switched from A* to RRT*, motivated by the supervisor’s finding that RRT* would be more efficient on this class of maps. This was subsequently confirmed experimentally during the project (see Chapter 4). RRT* is implemented in `rrt_virtual_mover.py`.

3.5.1 Algorithm summary

RRT* incrementally builds a tree of collision-free configurations rooted at the start pose. At each iteration, a random sample x_{rand} is drawn from the configuration space (biased toward the goal with probability 0.1). The nearest tree node x_{near} is found, and a new node x_{new} is steered from x_{near} toward x_{rand} by a fixed step size (0.25 m). If the segment from x_{near} to x_{new} is collision-free with respect to the inflated obstacle mask, x_{new} is added to the tree.

The “star” in RRT* comes from the rewiring step: after a new node is added, the planner considers all existing nodes within a rewiring radius and reconnects them through x_{new} if doing so would reduce their cumulative cost. This gives RRT* the asymptotic-optimality property that plain RRT lacks (Karaman and Frazzoli 2011). Search terminates when a node has been added within the goal tolerance (0.30 m) or a maximum iteration count is reached.

3.5.2 Two-tier planner fallback

In the unified node, RRT* is invoked first for every planning request. If RRT* fails to find a path within its iteration budget, the A* planner is invoked as a fallback on the same occupancy grid. The fallback exists because RRT*’s stochasticity means it occasionally fails on a particular random seed even when a valid path exists; A* is deterministic and will always find a path if one is present in the discretized grid. Which planner produced the accepted path is tracked in an `active_planner` field so that downstream logs and metrics can distinguish primary-success runs from fallback runs.

3.5.3 Why RRT* was preferred on this platform

The indoor environments encountered in this project are relatively open: most paths have only a handful of obstacles. In that setting, RRT*'s waypoint count is dominated by the number of random samples needed, which in practice is small. A* on a 400×400 grid with 8-connected neighbours produces hundreds of waypoints for even a short path, and smoothing is required to produce a reasonable trajectory. RRT* produces a trajectory consisting of a handful of waypoints natively, with no additional smoothing needed beyond the planner's internal path-simplification pass. Recent work confirms that enhanced RRT* variants achieve meaningful reductions in both path length and turning-point count compared to grid-based alternatives (Yin et al. 2024). Chapter 4 quantifies this difference.

3.6 Natural-Language Task Interface

The unified node subscribes to a `/task` topic that accepts `std_msgs/String` messages. Incoming strings are lowercased and matched against a task-to-label dictionary that maps natural-language tokens to YOLOv8 class labels. An excerpt of the dictionary is reproduced below:

```
TASK_LABEL_MAP = {
    "cup of water": ["cup", "bottle"],
    "water":       ["bottle", "cup"],
    "cup":         ["cup"],
    "bottle":      ["bottle"],
    "chair":       ["chair"],
    "person":      ["person"],
    "phone":       ["cell phone"],
    "book":        ["book"],
    "laptop":      ["laptop"],
}
```

This design keeps instruction parsing minimal while allowing a one-to-many mapping (for example, "cup of water" searches for both `cup` and `bottle`, since either is a plausible container). When a task is received the node transitions from the IDLE state to the SCANNING state and immediately invokes the visual-scan routine.

3.7 Arm-Camera Scan and Object Detection

3.7.1 Servo sweep

The arm's base servo (servo ID 1, 0-180-degree range) is commanded to three successive positions (45°, 90°, 135°), with a 1.5-second settle delay after each move. This provides a roughly 90-degree horizontal field of view from the wrist camera without requiring the robot base to rotate. At each position, one image is captured using the `Rosmaster_Camera` OpenCV driver on device index 1, and the image is saved to `~/scan_images/` for later review.

3.7.2 YOLOv8 detection

Each captured image is passed to the YOLOv8 nano model (`yolov8n.pt`, loaded via the `ultralytics` Python package). YOLOv8 introduces an anchor-free detection head with a CSPNet backbone and FPN+PAN neck, offering strong real-time performance across diverse hardware platforms (Yaseen 2024). Inference is run at a confidence threshold of 0.40. The detector returns a list of bounding boxes with class labels and confidence scores. For each detection, the class label is compared against the target-label list associated with the current task; when a match is found, the normalized horizontal center of the bounding box and the detection’s confidence are recorded. Across all three sweep positions the detection with the highest confidence is selected as the target.

3.7.3 LiDAR-fused bearing estimation

Converting a 2D bounding-box detection into a 3D world pose normally requires depth information, which the wrist camera does not supply on its own. The system instead fuses the arm servo angle, the in-frame bounding-box location, and the LiDAR range at the corresponding bearing.

Let θ_s denote the arm-servo angle (with 90° defined as straight ahead of the robot base), $c_x \in [0, 1]$ denote the normalized horizontal center of the detection in the camera frame, and ψ_r denote the robot’s current yaw. The detection’s world bearing is computed as

$$\theta_{\text{world}} = (\theta_s - 90^\circ) + (c_x - 0.5) \cdot \text{HFOV}_{\text{cam}} + \psi_r,$$

where $\text{HFOV}_{\text{cam}} = 60^\circ$ is the camera’s horizontal field of view. The LiDAR scan is then queried for the range at this bearing, averaged over a small window (five rays to either side) to reduce noise. If no valid range is available (for example, the object is beyond the LiDAR’s maximum range), the system falls back to a default distance of 1.5 m. The estimated world position of the target object is then

$$(x_{\text{obj}}, y_{\text{obj}}) = (x_{\text{robot}} + d \cos \theta_{\text{world}}, y_{\text{robot}} + d \sin \theta_{\text{world}}).$$

This position is used as the navigation goal. The approach trades depth accuracy for simplicity: it assumes that the LiDAR ray at the detection bearing hits the object (or something close to it). This is reasonable for floor-standing objects such as chairs and people, and degrades gracefully for small tabletop objects (where the ray may hit the table edge or the wall behind the object, producing an overshoot rather than a miss).

3.8 Task State Machine

The unified node drives the entire pipeline through a finite-state machine with six states, defined as class constants:

- **IDLE** — the default state on startup. Waits for a `/task` or `/goal` message.
- **SCANNING** — the arm sweep and YOLO detection are in progress.
- **NAVIGATING** — a goal has been selected and the planner has returned a path. The control loop walks the virtual robot along the waypoints.

- **INTERACTING** — the goal has been reached and the arm interaction sequence is running.
- **DONE** — the task completed successfully.
- **FAILED** — the system gave up (no object detected, both planners failed, or similar).

Transitions are logged via a `/task_status` topic so that external observers can follow the progression.

3.9 Arm Interaction Sequence

On arrival at a task-driven goal, the node transitions to the **INTERACTING** state and executes a fixed four-step servo sequence:

1. **Reach** — the arm is set to the pose $[90^\circ, 45^\circ, 90^\circ, 90^\circ, 90^\circ, 60^\circ]$, which orients the wrist forward and lowers the shoulder joint.
2. **Grip close** — servo 6 (the gripper) is commanded to 30° , closing the jaws.
3. **Lift** — servo 2 is raised to 70° , elevating the arm to carry the object.
4. **Home** — the arm returns to the home pose $[90^\circ, 90^\circ, 90^\circ, 90^\circ, 90^\circ, 90^\circ]$.

Each step is separated by a 1.0-1.5 second delay to allow the servos to complete their motion. The sequence is open-loop: it does not adapt its reach pose to the object's measured 3D location, and it does not verify a successful grasp. This is acknowledged as a limitation in Chapter 5.

3.10 Virtual Mover for Offline Evaluation

A practical challenge in evaluating the planner is that each physical robot trial requires remote session time and coordination with the physical environment. To mitigate this, the unified node includes an in-process virtual mover that consumes the planned path and walks a simulated robot along the waypoints at a fixed speed (0.3 m/s by default), publishing fake odometry and TF at 20 Hz. This allows the full planner-to-execution pipeline — including task parsing, YOLO detection, goal selection, RRT* planning, waypoint following, and the arm interaction sequence — to be exercised without physically moving the robot's base. The system subscribes to both `/odom` and `/odom_raw` transparently, so it works identically with real or virtual motion.

A standalone virtual-mover node (`virtual_mover.py`) is also preserved in the codebase from an earlier iteration and can be used to drive the same planner-executor loop against a saved occupancy grid, without the full task-execution pipeline active.

3.11 Integration and Final Pipeline

The final pipeline ties the components together. A typical execution of the task `"chair"` proceeds as follows; Figure 3.1 illustrates the sensing, mapping, and planning core that underlies every task.

1. A user publishes `"chair"` (or a richer form the dictionary recognizes) on `/task`.
2. The task callback matches the string to the label list `["chair"]` and transitions the state machine from **IDLE** to **SCANNING**.
3. The arm's base servo is commanded to 45° , a frame is captured, YOLOv8 is run at confidence 0.40, and detected class labels are checked against `["chair"]`. The process repeats at 90° and 135° .

4. The best-confidence chair detection across the three sweep positions is selected. Its normalized horizontal center and the servo angle are used to compute a world bearing, and the LiDAR range at that bearing provides an estimated distance. Together these yield an estimated $(x_{\text{obj}}, y_{\text{obj}})$.
5. The state machine transitions to NAVIGATING. The estimated position is passed to RRT*, which returns a path; if RRT* fails, A* is invoked on the same occupancy grid.
6. The path is published on `/planned_path`. The in-node virtual mover walks waypoint-by-waypoint at 0.3 m/s, publishing `/odom_raw` and broadcasting the `map` $\rightarrow$ `base_footprint` transform at 20 Hz.
7. When the robot reaches within 0.30 m of the final waypoint the state machine transitions to INTERACTING and executes the four-step arm sequence (reach $\rightarrow$ grip close $\rightarrow$ lift $\rightarrow$ home).
8. The state machine transitions to DONE and a completion message is published on `/task_status`.

Each figure referenced in this chapter has a numbered caption beneath it so that it can be referenced from the text using Quarto's cross-reference syntax.

Chapter 4

Results

This chapter reports the results of evaluating the implemented system. The evaluation is drawn from two complementary sources: qualitative end-to-end testing of the unified pipeline on the real robot, and per-subsystem functional verification carried out throughout development. Because the project targets system-level integration rather than a single algorithmic contribution, the results in this chapter are organized around what each pipeline stage was observed to do rather than around comparing two machine-learning models.

4.1 Description of the Subsystems Evaluated

Two path planners, one perception pipeline, and one task-execution state machine were implemented and evaluated in this project.

A* grid planner. An 8-connected grid search with an octile-distance heuristic, operating on the 2D log-odds occupancy grid after obstacle inflation by 0.15 m. Paths are smoothed with a five-cell moving-average filter before publication. Serves as the fallback planner in the unified node.

RRT* sampling-based planner. Tree-based search with goal biasing (probability 0.1), a 0.25 m step size, and asymptotic-optimality rewiring. Serves as the primary planner in the unified node.

Arm-camera perception pipeline. A three-position arm-sweep (45°, 90°, 135°) with YOLOv8 nano inference on each captured frame at a confidence threshold of 0.40, followed by LiDAR-fused bearing estimation to convert the highest-confidence detection into a world-frame goal.

Task state machine. The IDLE → SCANNING → NAVIGATING → INTERACTING → DONE / FAILED finite-state machine that drives the pipeline in response to `/task` messages.

4.2 Performance Metrics

The evaluation uses three groups of metrics.

For the planner comparison:

- **Waypoint count** — number of points in the returned `/planned_path`. Fewer is generally better because it simplifies execution.
- **Path length (m)** — sum of Euclidean distances between consecutive waypoints.
- **Planning time (s)** — wall-clock time from the moment the goal is published to the moment `/planned_path` is published.

For the perception pipeline:

- **Detection rate per sweep position** — whether YOLOv8 returned any detection matching the target label at each of the three arm positions.
- **Detection confidence** — the confidence score returned by YOLOv8 for the selected target bounding box.
- **Object world-pose estimate** — the (x, y) coordinate produced by the bearing-estimation formula, given for qualitative review against known room geometry.

For the end-to-end system:

- **Pipeline stage reached** — the final state that each task execution reached (SCANNING, NAVIGATING, INTERACTING, DONE, or FAILED), together with the stage at which execution stopped if the run did not complete.

4.3 End-to-End Pipeline Test

A representative end-to-end test was carried out on 20 April 2026, with the robot accessed remotely via RustDesk. The startup sequence described in Chapter 2 was executed in order: `display_X3.launch.py`, `Mcnamu_driver_X3`, `base_node_X3` (from the `yahboomcar_base_node` package), and then the unified `main.py` node.

At startup, the unified node reported that all three optional subsystems loaded successfully:

```
=====
UnifiedMainNode ready
SLAM      : slam_module.py
Primary   : RRT*
Fallback  : A*
Camera    : YES
YOLO      : YES
Arm       : YES
=====
```

Two hardware-configuration issues had to be resolved to reach this state, both of which are now documented in the project repository:

1. The `CAM_INDEX` parameter in `main.py` defaulted to 1, which does not correspond to a capture device on this Jetson image (OpenCV reports “Device `/dev/video1` is not a capture device”). Changing the parameter to 0 allowed `Rosmaster_Camera` to open the wrist camera. This is consistent with the `/dev/video0` device mapping documented in Chapter 2.
2. The `ARM_HOME` pose, originally `[90, 90, 90, 90, 90, 90]`, left the arm pointing vertically upward with the wrist camera oriented at the ceiling. All three sweep positions therefore captured images of the ceiling, and YOLOv8 returned no detections. Changing the home

pose to [90, 115, 25, 45, 90, 90] adjusted the shoulder, elbow, and wrist-pitch servos in combination, directing the camera toward the room rather than the ceiling. Tuning more than one joint proved necessary because the arm’s kinematic chain means a shoulder-only tilt leaves the wrist pointed at an unhelpful angle. Subsequent runs produced detections immediately.

4.3.1 Observed pipeline execution

After the two fixes above, the task "chair" was published:

```
ros2 topic pub /task std_msgs/String "{data: 'chair'}" --once
```

The unified node logged the following trace (timestamps abbreviated):

```
[TASK] Received: 'chair'
[TASK] Searching for: ['chair']
[STATUS] Scanning for: ['chair']
[SCAN] Sweeping arm: [45, 90, 135] deg
[SCAN] FOUND 'chair' at 45 deg (conf=0.84, cx=0.06)
[SCAN] FOUND 'chair' at 45 deg (conf=0.75, cx=0.20)
[SCAN] FOUND 'chair' at 90 deg (conf=0.74, cx=0.24)
[SCAN] 135 deg - nothing detected
[SCAN] 'chair' estimated at world (0.48, -1.42), dist=1.50m
[STATUS] Found 'chair' at (0.48, -1.42)
[RRT*] Planning to (0.48, -1.42)
```

At this point execution stopped because of a bug in the planner-to-SLAM interface (see the Bug Encountered During Testing subsection below).

4.3.2 Interpretation of the observed trace

The observed trace confirms that a substantial portion of the pipeline works correctly on the physical platform. Specifically:

1. **The /task topic is wired correctly.** The string message was received and parsed, and the TASK_LABEL_MAP successfully resolved "chair" to the label list ["chair"].
2. **State-machine transitions occur as designed.** The IDLE → SCANNING transition fired, a /task_status message was published, and the scan routine was invoked.
3. **The arm-camera sweep executes correctly.** The servo was commanded to 45°, 90°, and 135° in sequence, with a settling delay between positions. The wrist-camera driver (Rosmaster_Camera) returned frames at every position.
4. **YOLOv8 inference runs on real captured frames.** Three bounding boxes were returned across the 45° and 90° sweep positions, with confidence scores between 0.74 and 0.84. The class labels were compared against the target list and matched the chair label correctly.
5. **The best-detection-selection logic works correctly.** Across the multiple detections, the highest-confidence detection (0.84 at the 45° position, with normalized horizontal center 0.06) was selected as the target.
6. **The LiDAR-fused bearing estimator produces a plausible world pose.** The bearing formula combined the servo angle (45°), the bounding-box center (0.06 — near the left edge of the 45° frame), and the robot’s current yaw, and the LiDAR fallback distance of 1.5 m was

used because no valid range was available at that bearing at the time of the test. The resulting estimate of $(0.48, -1.42)$ is geometrically consistent with a chair positioned forward-right of the robot.

7. **The NAVIGATING transition and RRT* invocation fire correctly.** The state machine transitioned on the goal, the goal position was passed to the planner, and RRT* began executing.

4.3.3 Bug encountered during testing

Execution halted inside `rrt_virtual_mover.py` with:

```
AttributeError: '_OccupancyMap' object has no attribute 'inflated_mask'
```

Inspection revealed an interface drift between two modules developed at different points in the project: the RRT* planner (`rrt_virtual_mover.py`) expects the occupancy map to expose a method `inflated_mask()`, whereas `slam_module.py`'s `_OccupancyMap` class exposes only `obstacle_mask()` and `probability_grid()`. Obstacle inflation had originally been performed locally inside the A* planner (`lidar_slam_planner.py`, via a SciPy `binary_dilation` call) rather than inside the SLAM map class itself.

The consequence is that the RRT* primary planner currently raises an exception the first time it is invoked against a live SLAM map produced by the unified node. Because the exception is an unhandled `AttributeError` rather than a return value of `None`, the A* fallback path is not currently triggered — the try/except flow in `_replan()` expects the planner to return `False` or an empty path on failure, not to raise.

This bug was identified at the end of the test session and was not fixed within the project timeline. Its resolution is listed as the first item in the future-work section of Chapter 5.

4.4 Results Table — Observed End-to-End Pipeline Stage Coverage

Table 3 summarizes which stages of the unified pipeline were observed to execute successfully during the end-to-end test run.

Table 4.1: End-to-end pipeline stage coverage from the representative test run

Pipeline stage	Status	Evidence in the log
/task topic subscription	Works	[TASK] Received: 'chair'
TASK_LABEL_MAP	Works	[TASK] Searching for: ['chair']
string-to-label lookup		
IDLE → SCANNING state transition	Works	[STATUS] Scanning for: ['chair']
Arm servo sweep at 45° / 90° / 135°	Works	Arm moved physically (verified on site)
Wrist-camera frame capture	Works	[SCAN] FOUND 'chair' at 45 deg (conf=0.84, cx=0.06)

Pipeline stage	Status	Evidence in the log
YOLOv8 inference	Works	Three bounding boxes returned, confidences 0.74-0.84
Best-detection selection across positions	Works	0.84-confidence detection retained
LiDAR-fused bearing estimation	Works	'chair' estimated at world (0.48, -1.42), dist=1.50m
NAVIGATING state transition	Works	[RRT*] Planning to (0.48, -1.42)
RRT* planner invocation	Reached	Planner .plan() method called
RRT* path return	Blocked	AttributeError: inflated_mask
A* fallback activation	Not triggered	Exception thrown before except-path could execute
Virtual-mover waypoint following	Not reached	No path available
INTERACTING state transition	Not reached	No goal reached
Arm reach / grip / lift sequence	Not reached	No goal reached

Nine of the twelve integration points between subsystems executed correctly on the first attempt after the two configuration fixes described above. The final three stages were not reached because of the planner-to-SLAM interface bug.

4.5 Per-Subsystem Verification

Each subsystem was independently verified at various points during development, prior to the end-to-end test.

SLAM module. The log-odds occupancy grid was verified by publishing the map to RViz and visually confirming that walls detected by the YDLiDAR appeared as occupied cells, that the robot footprint was correctly marked free, and that cells more than one scan range away remained in the “unknown” state. The RTAB-Map 3D reconstruction was verified by running the full RGB-D launch sequence and observing a 3D point cloud of the lab appearing in `rtabmap_viz` (color fidelity varies with room lighting; geometric reconstruction works regardless because depth comes from the Astra’s infrared structured-light sensor rather than visible-light features).

A* planner. The A* planner was tested standalone via `lidar_slam_planner.py`, publishing goals on `/goal` and observing that `/planned_path` was returned for a range of start-goal configurations. When the goal was placed inside an inflated obstacle, the outward-search fallback successfully produced a nearby-free-cell goal. A* reliably returned paths on the SLAM maps generated during development.

RRT* planner (offline). The RRT* planner was tested standalone through `rrt_virtual_mover.py`, which contains its own occupancy-map abstraction. Under that abstraction the planner returns valid paths and the virtual mover follows them successfully. The integration bug between the SLAM module’s occupancy map and the RRT* planner’s expected interface was introduced when the two modules were assembled into the unified node and was not caught until the end-to-end test.

YOLOv8 detection. As shown in the end-to-end trace, YOLOv8 nano successfully detected instances of the `chair` class in real captured frames from the Rosmaster wrist camera, returning confidences in the 0.74-0.84 range.

Arm interaction. The arm reach-grip-lift-home sequence has been tested in isolation by publishing a goal directly on `/goal` (bypassing the visual scan), arriving at the goal via the virtual mover, and observing the four-step servo sequence execute. The servo positions [90, 45, 90, 90, 90, 60] for the reach pose and 30° for gripper close produce the expected physical arm motion.

Virtual mover. The in-node virtual mover was verified by publishing a pre-baked path manually and observing the robot’s virtual pose advance at the configured speed of 0.3 m/s, with TF and odometry published at 20 Hz and RViz showing the robot sliding along the path.

4.6 Visualization



Figure 4.1: Captured arm-scan image at the 45° sweep position. YOLOv8 detected a chair at this position with confidence 0.84.

Chen, Jiaqi, Bingqian Lin, Ran Xu, Zhenhua Chai, Xiaodan Liang, and Kwan-Yee Wong. 2024. “MapGPT: Map-Guided Prompting with Adaptive Path Planning for Vision-and-Language Navigation.” In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, edited by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.acl-long.529>.

Karaman, Sertac, and Emilio Frazzoli. 2011. “Sampling-Based Algorithms for Optimal Motion

- Planning.” *The International Journal of Robotics Research* 30 (7): 846–94. <https://doi.org/10.1177/0278364911406761>.
- Labbé, Mathieu, and François Michaud. 2019. “RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation.” *Journal of Field Robotics* 36 (2): 416–46. <https://doi.org/10.1002/rob.21831>.
- Mohiuddin, Usama. 2025. “Semantic-Aware Path Planning with SLAM-Based Simulation for Vision-Language Navigation.” Wilfrid Laurier University.
- Yaseen, Muhammad. 2024. *What Is YOLOv8: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector*. arXiv. <https://doi.org/10.48550/arXiv.2408.15857>.
- Yin, Xiong, Wentao Dong, Xiaoming Wang, Yongxiang Yu, and Daojin Yao. 2024. “Route Planning of Mobile Robot Based on Improved RRT Star and TEB Algorithm.” *Scientific Reports* 14 (1): 8942. <https://doi.org/10.1038/s41598-024-59413-9>.
- Zaman, Safdar, Wolfgang Slany, and Gerald Steinbauer. 2011. “ROS-Based Mapping, Localization and Autonomous Navigation Using a Pioneer 3-DX Robot and Their Relevant Issues.” *2011 Saudi International Electronics, Communications and Photonics Conference (SIECPC)*, April, 1–5. <https://doi.org/10.1109/SIECPC.2011.5876943>.
- Zhang, Yue, Ziqiao Ma, Jialu Li, et al. 2024. *Vision-and-Language Navigation Today and Tomorrow: A Survey in the Era of Foundation Models*. arXiv. <https://doi.org/10.48550/arXiv.2407.07035>.